

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Department	Computer Science Engineering
Programme	B. Tech
Course Code & Course Name	CSA0924 – Programming in Java
Academic Year / Batch	2024
Faculty Name	Dr. S. Padmakala
Assignment Title	Real-Time Concurrent Bank Fund-Transfer & Ledger System — Preventing the “Double-Spend / Lost-Update” Bug
Date of Issue	01.09.2026
Date of Submission	01.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO1, CO2, CO3, CO5 (core) and CO4 (bonus)
Bloom's Taxonomy Level	L3–L6 (Apply, Analyze, Evaluate, Create)
SDG Mapping (SDG 1-17)	SDG 8 – Decent Work and Economic Growth (resilient financial infrastructure)
Industry / Societal Relevance	Lost-update and double-spend defects caused by unsynchronized concurrent access to shared balances are real, recurring production incidents in banking, fintech, and payment-gateway systems; this assignment reproduces the exact class of concurrency defect financial-systems engineers are hired to prevent.

Real-Time Concurrent Bank Fund-Transfer & Ledger System

1. Problem Understanding

The banking system must process multiple withdrawal and transfer requests concurrently. If balance checking and debit operations are not synchronized, two threads may access the same balance simultaneously, causing a **lost-update/double-spend problem**.

Transfers can also cause **deadlock** when two threads lock accounts in opposite orders.

The solution must ensure:

- No negative balances
- No lost updates
- No deadlock
- Failed transfers are rolled back

- Successful transactions are stored in the database

These are the core requirements specified in the assignment.

2. OOP Classes

Customer

```
class Customer {  
    private String customerId;  
    private String name;  
  
    public Customer(String customerId, String name) {  
        this.customerId = customerId;  
        this.name = name;  
    }  
  
    public String getCustomerId() {  
        return customerId;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Account

```
abstract class Account {  
    private String accountNumber;  
    private Customer customer;  
    protected double balance;  
    private boolean frozen;  
  
    public Account(String accountNumber,  
                   Customer customer,  
                   double balance) {  
        this.accountNumber = accountNumber;  
        this.customer = customer;  
        this.balance = balance;  
    }  
}
```

```

        this.frozen = false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public synchronized double getBalance() {
        return balance;
    }

    public boolean isFrozen() {
        return frozen;
    }

    public void freeze() {
        frozen = true;
    }

    public void unfreeze() {
        frozen = false;
    }

    public abstract void applyInterestOrFee();
}

```

SavingsAccount

```

class SavingsAccount extends Account {

    public SavingsAccount(String accountNumber,
                           Customer customer,
                           double balance) {
        super(accountNumber, customer, balance);
    }

    @Override
    public void applyInterestOrFee() {
        synchronized (this) {
            balance += balance * 0.04;
        }
    }
}

```

```
    }  
}
```

CurrentAccount

```
class CurrentAccount extends Account {  
  
    public CurrentAccount(String accountNumber,  
                           Customer customer,  
                           double balance) {  
        super(accountNumber, customer, balance);  
    }  
  
    @Override  
    public void applyInterestOrFee() {  
        synchronized (this) {  
            balance -= 50;  
        }  
    }  
}
```

Transaction

```
class Transaction {  
    private String id;  
    private String type;  
    private String from;  
    private String to;  
    private double amount;  
  
    public Transaction(String id, String type,  
                       String from, String to,  
                       double amount) {  
        this.id = id;  
        this.type = type;  
        this.from = from;  
        this.to = to;  
        this.amount = amount;  
    }  
  
    @Override
```

```

    public String toString() {
        return id + " | " + type + " | ₹" + amount;
    }
}

```

3. Custom Exceptions

```

class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) {
        super(msg);
    }
}

```

```

class InvalidAccountException extends Exception {
    public InvalidAccountException(String msg) {
        super(msg);
    }
}

```

```

class AccountFrozenException extends Exception {
    public AccountFrozenException(String msg) {
        super(msg);
    }
}

```

```

class TransactionLimitExceededException extends Exception {
    public TransactionLimitExceededException(String msg) {
        super(msg);
    }
}

```

4. Bank Using Collections

```

import java.util.*;

class Bank {

    private Map<String, Account> accounts =

```

```

        new HashMap<>();

private List<Transaction> ledger =
        new ArrayList<>();

public void addAccount(Account account) {
    accounts.put(account.getAccountNumber(), account);
}

public Account getAccount(String number)
        throws InvalidAccountException {

    Account account = accounts.get(number);

    if (account == null)
        throw new InvalidAccountException(
            "Invalid account: " + number);

    return account;
}

public void addTransaction(Transaction t) {
    ledger.add(t);
}

public void showLedger() {
    Iterator<Transaction> it = ledger.iterator();

    while (it.hasNext()) {
        System.out.println(it.next());
    }
}
}

```

5. Unsynchronized Withdrawal – Before

```

public static void unsafeWithdraw(
    Account account, double amount)
    throws InterruptedException {

    if (account.getBalance() >= amount) {

        Thread.sleep(100);

        account.balance -= amount;

        System.out.println(
            Thread.currentThread().getName()
            + " withdrew ₹" + amount
            + " Balance = ₹"
            + account.getBalance());
    }
}

```

Problem

Two threads can simultaneously execute:

```

Thread 1 → Check balance
Thread 2 → Check balance
Thread 1 → Debit
Thread 2 → Debit

```

This is the **lost-update/double-spend race condition**.

6. Thread-Safe Withdrawal – After

```

public synchronized void withdraw(double amount)
    throws InsufficientBalanceException,
        AccountFrozenException {

    if (frozen)
        throw new AccountFrozenException(
            "Account is frozen");
}

```

```
    if (balance < amount)
        throw new InsufficientBalanceException(
            "Insufficient balance");

    balance -= amount;
}
```

The check and debit now happen atomically.

7. Deadlock-Prone Transfer

```
synchronized (from) {

    synchronized (to) {

        from.balance -= amount;
        to.balance += amount;
    }
}
```

Example:

```
Thread 1: Lock A → Wait for B
Thread 2: Lock B → Wait for A
```

Both threads wait indefinitely.

8. Deadlock-Free Transfer

Accounts are always locked according to account-number order.

```
public static void transfer(
    Account from,
    Account to,
```



```

        double amount)
        throws Exception {

    Account first;
    Account second;

    if (from.getAccountNumber()
        .compareTo(to.getAccountNumber()) < 0) {

        first = from;
        second = to;

    } else {

        first = to;
        second = from;
    }

    synchronized (first) {

        synchronized (second) {

            if (from.isFrozen())
                throw new AccountFrozenException(
                    "Source account is frozen");

            if (to.isFrozen())
                throw new AccountFrozenException(
                    "Destination account is frozen");

            if (from.getBalance() < amount)
                throw new InsufficientBalanceException(
                    "Insufficient balance");

            from.balance -= amount;

            try {

                to.balance += amount;

            } catch (Exception e) {

```

```

        from.balance += amount;

        throw e;
    }
}
}
}

```

9. 20-Thread Stress Test

```

import java.util.concurrent.*;

public class BankTest {

    public static void main(String[] args)
        throws Exception {

        Customer c =
            new Customer("C001", "Anish");

        Account account =
            new SavingsAccount(
                "A1001", c, 10000);

        ExecutorService executor =
            Executors.newFixedThreadPool(20);

        for (int i = 1; i <= 20; i++) {

            final int id = i;

            executor.submit(() -> {

                try {

                    account.withdraw(1000);

                    System.out.println(

```

```

        "Thread-" + id +
        " SUCCESS | Balance = ₹"
        + account.getBalance());

    } catch (Exception e) {

        System.out.println(
            "Thread-" + id +
            " FAILED | "
            + e.getMessage());
    }
});
}

executor.shutdown();

executor.awaitTermination(
    10, TimeUnit.SECONDS);

System.out.println(
    "Final Balance = ₹"
    + account.getBalance());
}
}

```

The assignment requires at least **20 concurrent threads** for the stress test.

Expected Output

```

Thread-1 SUCCESS | Balance = ₹9000
Thread-2 SUCCESS | Balance = ₹8000
Thread-3 SUCCESS | Balance = ₹7000
Thread-4 SUCCESS | Balance = ₹6000
Thread-5 SUCCESS | Balance = ₹5000
Thread-6 SUCCESS | Balance = ₹4000
Thread-7 SUCCESS | Balance = ₹3000
Thread-8 SUCCESS | Balance = ₹2000
Thread-9 SUCCESS | Balance = ₹1000
Thread-10 SUCCESS | Balance = ₹0

Thread-11 FAILED | Insufficient balance

```

```
Thread-12 FAILED | Insufficient balance
...
Thread-20 FAILED | Insufficient balance

Final Balance = ₹0
```

10. JDBC Database

Accounts Table

```
CREATE TABLE accounts (
    account_no VARCHAR(20) PRIMARY KEY,
    customer_id VARCHAR(20),
    balance DECIMAL(15,2),
    frozen BOOLEAN
);
```

Transactions Table

```
CREATE TABLE transactions (
    transaction_id VARCHAR(30) PRIMARY KEY,
    transaction_type VARCHAR(20),
    from_account VARCHAR(20),
    to_account VARCHAR(20),
    amount DECIMAL(15,2),
    transaction_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

11. JDBC Transfer

```
Connection con =
    DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/bankdb",
        "root",
        "root");
```

```

con.setAutoCommit(false);

try {

    String sql =
        "INSERT INTO transactions " +
        "(transaction_id, transaction_type, " +
        "from_account, to_account, amount) " +
        "VALUES (?, ?, ?, ?, ?)";

    PreparedStatement ps =
        con.prepareStatement(sql);

    ps.setString(1, "T001");
    ps.setString(2, "TRANSFER");
    ps.setString(3, "A1001");
    ps.setString(4, "A1002");
    ps.setDouble(5, 2000);

    ps.executeUpdate();

    con.commit();

} catch (Exception e) {

    con.rollback();
}

```

The assignment requires JDBC persistence and `PreparedStatement`, with rollback when a transaction fails.

12. Audit Log

```

import java.io.*;

class AuditLog {

    public static synchronized void write(
        String message) {

```

```

        try (FileWriter fw =
            new FileWriter(
                "bank_audit.log", true)) {

            fw.write(
                message +
                System.lineSeparator());

        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Example:

```

T001 | TRANSFER | A1001 → A1002 | ₹2000 | SUCCESS
T002 | WITHDRAW | A1001 | ₹1000 | SUCCESS
T003 | WITHDRAW | A1001 | ₹50000 | FAILED

```

13. Final Results

Test	Result
Unsynchronized withdrawal	Lost update possible
Synchronized withdrawal	Correct
Negative balance	Prevented
Concurrent 20 threads	Passed
Naive transfer	Deadlock possible
Ordered transfer	Deadlock prevented
Insufficient balance	Exception generated
Frozen account	Exception generated
Invalid account	Exception generated
Failed transfer	Rollback performed
JDBC transaction	Stored
Audit log	Generated

Conclusion

The redesigned banking system solves the major concurrency problems by making balance operations atomic and acquiring two account locks in a consistent order. Custom exceptions handle transaction failures, rollback maintains consistency, and JDBC provides persistent transaction storage. This satisfies the essential implementation and validation requirements of the assignment.

FINAL OUTPUT

```
=====
REAL-TIME CONCURRENT BANKING SYSTEM
=====
Account added: A1001
Account added: A1002

===== ACCOUNTS =====
A1002 | Rahul | Balance = ₹10000.00
A1001 | Anish | Balance = ₹10000.00

Performing deposit...
Deposit successful.

Performing withdrawal...
Withdrawal successful.

Performing transfer...
Transfer successful.

=====
20-THREAD CONCURRENT WITHDRAWAL TEST
=====
Thread-1 SUCCESS | Balance = ₹8500.00
Thread-3 SUCCESS | Balance = ₹6500.00
Thread-7 SUCCESS | Balance = ₹2500.00
Thread-2 SUCCESS | Balance = ₹7500.00
Thread-6 SUCCESS | Balance = ₹3500.00
Thread-4 SUCCESS | Balance = ₹5500.00
Thread-5 SUCCESS | Balance = ₹4500.00
Thread-8 SUCCESS | Balance = ₹1500.00
Thread-9 SUCCESS | Balance = ₹500.00
Thread-10 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-16 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-18 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-17 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-20 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-19 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-15 FAILED | Insufficient balance in account A1001. Available = ₹500.00
```

Thread-11 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-13 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-14 FAILED | Insufficient balance in account A1001. Available = ₹500.00
Thread-12 FAILED | Insufficient balance in account A1001. Available = ₹500.00

Final Balance of A1001 = ₹500.00

=====
CONCURRENT BIDIRECTIONAL TRANSFER TEST
=====

Thread-3 A1002 -> A1001 SUCCESS
Thread-5 A1002 -> A1001 SUCCESS
Thread-6 A1001 -> A1002 SUCCESS
Thread-2 A1001 -> A1002 SUCCESS
Thread-4 A1001 -> A1002 SUCCESS
Thread-1 A1002 -> A1001 SUCCESS
Thread-8 A1001 -> A1002 SUCCESS
Thread-7 A1002 -> A1001 SUCCESS
Thread-9 A1002 -> A1001 SUCCESS
Thread-12 A1001 -> A1002 SUCCESS
Thread-11 A1002 -> A1001 SUCCESS
Thread-10 A1001 -> A1002 SUCCESS
Thread-14 A1001 -> A1002 SUCCESS
Thread-13 A1002 -> A1001 SUCCESS
Thread-15 A1002 -> A1001 SUCCESS
Thread-17 A1002 -> A1001 SUCCESS
Thread-16 A1001 -> A1002 SUCCESS
Thread-19 A1002 -> A1001 SUCCESS
Thread-20 A1001 -> A1002 SUCCESS
Thread-18 A1001 -> A1002 SUCCESS

All transfer threads completed.
A1001 Balance = ₹500.0
A1002 Balance = ₹11000.0

=====
ROLLBACK TEST
=====

Balance before failed transfer: ₹500.0
Transfer failed: Account not found: INVALID
Balance after rollback: ₹500.0

===== ACCOUNTS =====
A1002 | Rahul | Balance = ₹11000.00
A1001 | Anish | Balance = ₹500.00

===== TRANSACTION LEDGER =====
T2394733086133 | DEPOSIT | From: - | To: A1001 | Amount: ₹1000.00 | SUCCESS
T2394850375149 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹500.00 | SUCCESS
T2394851351109 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹1000.00 | SUCCESS
T2394865556512 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 | SUCCESS
T2394868049548 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 | SUCCESS

T2394868837522 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 |
SUCCESS
T2394872154689 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 |
SUCCESS
T2394875612389 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 |
SUCCESS
T2394876235858 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 |
SUCCESS
T2394879778359 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 |
SUCCESS
T2394881275364 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 |
SUCCESS
T2394883439380 | WITHDRAWAL | From: A1001 | To: - | Amount: ₹1000.00 |
SUCCESS
T2394903390641 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394903957925 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394904987722 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394907799375 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394908904769 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394909958174 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394912867737 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394914210585 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394914791329 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394915258460 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394915893189 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394916498771 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394917238671 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394917934832 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394918754763 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394919458867 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394919816135 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394927288758 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS
T2394927806853 | TRANSFER | From: A1002 | To: A1001 | Amount: ₹100.00 |
SUCCESS
T2394928405910 | TRANSFER | From: A1001 | To: A1002 | Amount: ₹100.00 |
SUCCESS

=====

SYSTEM TEST COMPLETED

=====

Audit log: bank_audit.log

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1	[Faculty to map]	L3/L4	10
Application of knowledge	CO1, CO2	[Faculty to map]	L3/L4	20
Solution / Design / Implementation	CO1, CO2, CO3	[Faculty to map]	L4/L5/L6	20
Modern tool usage (multithreading + JDBC)	CO3, CO5	[Faculty to map]	L3/L4	10
Validation	CO3, CO5	[Faculty to map]	L4/L5	15
Analysis & justification	CO3	[Faculty to map]	L4/L5	15
Broader considerations	CO1–CO5	[Faculty to map]	L4/L5	5
Documentation & reflection	CO1–CO5	[Faculty to map]	L3/L4	5

Note: PO mapping is institution/programme-specific and left for the faculty to complete based on the department's approved PO framework. CO mapping above is pre-filled based on the CSA09 syllabus Course Outcomes (CO1: OOP design; CO2: Collections Framework; CO3: Exception handling & multithreading; CO4: GUI/event handling — bonus part; CO5: JDBC connectivity).

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____